

PIC16F84 Simulator

Dokumentation

Studiengang Informatik

Duale Hochschule Baden-Württemberg Karlsruhe

Nico Rigsinger

Louis Wunsch

Eingereicht am: 17.05.2026

Matrikelnummer, Kurs: 6522771, TINF24B3
7579974, TINF24B3

Betreuer an der DHBW: Stefan Lehmann

Inhalt

1	Einleitung	1
1.1	Motivation und Zielsetzung	1
1.2	Abgrenzung des Simulationsumfangs	1
1.3	Aufbau der Dokumentation	2
2	Aufgabenstellung und Technologieauswahl	3
2.1	Anforderungen	3
2.2	Wahl der Programmiersprache	4
2.3	Wahl des GUI-Frameworks	4
3	Architektur des Simulators	5
3.1	Schichtenarchitektur	5
3.2	Zusammenspiel der Komponenten	6
4	Programm- und Datenspeicher	8
4.1	Programmspeicher	8
4.2	Einlesen und Parsen der LST-Datei	8
4.3	Datenspeicher und Bankumschaltung	9
4.4	Zustand nach dem Reset	10
5	Der Stack	11
6	Die Flags im Statusregister	13
6.1	Zero-Flag	13
6.2	Carry- und Digit-Carry-Flag bei der Addition	13
6.3	Das besondere Verhalten der Flags bei der Subtraktion	14
7	Der Programmzähler	15
7.1	Verhalten beim Fetch-Zyklus	15
7.2	Verhalten bei Sprungbefehlen	15
7.3	Manipulation des PCL-Registers	16
8	Befehlssatz, Dekoder und Adressierung	17
8.1	Der Befehlsdekoder	17
8.2	Das Destination-Bit	18
8.3	Unmittelbare Adressierung	19
8.4	Direkte Adressierung	19
8.5	Indirekte Adressierung	19

9	Die Ports und das I/O-Ausgangslatch	20
9.1	Das Ausgangslatch	20
9.2	Simulation der Eingänge	21
9.3	Sonderpins RA4, RB0 und RB4 bis RB7	21
10	Timer, Watchdog und Vorteiler	22
10.1	Timer 0	22
10.2	Der Vorteiler	23
10.3	Der Watchdog	23
11	Das Interruptsystem	25
11.1	Auslösebedingungen	25
11.2	Eintritt in die Interruptroutine	25
11.3	Erkennung der externen Flanken	26
12	EEPROM, Sleep und Laufzeit	27
12.1	Sleep-Modus	27
12.2	Laufzeitberechnung	28
13	Die grafische Benutzeroberfläche	29
13.1	Aufbau der Oberfläche	29
13.2	Steuerung der Simulation	30
13.3	Breakpoints und Befehlsmarkierung	30
13.4	Ausgabe über das Lauflicht	30
14	Fazit	31
14.1	Rückblick auf das Projekt	31
14.2	Herausforderungen	31
14.3	Was wir gelernt haben	32
14.4	Abschließende Bewertung	32
A	Literatur	33
B	Glossar	34



Einleitung

1.1 Motivation und Zielsetzung

Im Rahmen der Vorlesung „Systemnahe Programmierung“ an der Duale Hochschule Baden-Württemberg bestand die Aufgabe darin, einen vollständig funktionsfähigen Simulator für den Mikrocontroller PIC16F84 zu entwickeln. Ziel des Projektes war es, die interne Funktionsweise, die Registerarchitektur sowie den Befehlssatz des Mikrocontrollers detailliert in Software nachzubilden. Wenn der simulierte Controller ein Programm abarbeitet, soll an seinen virtuellen Ein- und Ausgängen exakt dasselbe Verhalten auftreten wie bei einem physischen Baustein dieses Typs.

Das übergeordnete Ziel des Projekts ist es, das Verständnis für die innere Funktionsweise eines Mikroprozessors zu vertiefen. So konnten die theoretischen Grundlagen der Vorlesung praktisch angewendet und erweitert werden.

1.2 Abgrenzung des Simulationsumfangs

Simuliert wird ausschließlich die logische Ebene des Controllers, nicht die elektrische. Effekte wie Einschwingverhalten, Schaltschwellen, Temperaturverhalten, Stromverbrauch oder Spannungsschwankungen sind für das Projekt ohne Bedeutung und werden bewusst vernachlässigt. Ebenfalls nicht nachgebildet werden der Power-Up-Timer, der Oszillator-Startup-Timer sowie die zweistufige Synchronisationsverzögerung des Timers, da diese Funktionen für das Verständnis der Rechnerarchitektur nicht zentral und im Detail nur schwer abzubilden sind.

Der Schwerpunkt liegt stattdessen auf der korrekten Nachbildung der Befehlsausführung, des Speicher- und Registermodells, der Statusflags, der Timer-/Watchdog-/Vorteiler-Logik, des Interruptsystems sowie der Ein-/Ausgabe über die Ports.

1.3 Aufbau der Dokumentation

Abschnitt 2 fasst die Aufgabenstellung und die gewählten Technologie-Entscheidungen zusammen. Abschnitt 3 beschreibt die Gesamtarchitektur des Simulators. Abschnitt 4 - Abschnitt 12 erläutern die einzelnen Komponenten des Simulators im Detail. Abschnitt 13 stellt die Benutzeroberfläche vor. Abschließend wird in Abschnitt 14 ein persönliches Fazit gezogen.



Aufgabenstellung und Technologieauswahl

2.1 Anforderungen

Aus Themenblatt und Bewertungsschema ergeben sich folgende Kernanforderungen an den Simulator:

- Einlesen einer Programmdatei im LST-Format, Parsen der relevanten Spalten und Ablage im Programmspeicher.
- Vollständige Nachbildung des Befehlssatzes des PIC16F84, inklusive korrekter Behandlung der Statusflags Z, C und DC.
- Programm- und Datenspeicher als adressierbare Felder, inklusive Bankumschaltung und Adressspiegelung.
- Acht-stufiger Stack als Ringpuffer für Unterprogrammaufrufe.
- Timer 0 mit Vorteiler, Watchdog und korrekter Vorteilerzuweisung.
- Interruptsystem für Timer 0, den externen INT-Eingang (RB0) sowie die Pegelwechsel-Interrupts an RB4 bis RB7.
- Korrekte Abbildung des Programmzählers inklusive PCLATH bei Sprungbefehlen und bei direkter Manipulation des PCL-Registers.
- Grafische Benutzeroberfläche mit Bedienelementen, Registeranzeige, Quelltextansicht mit Markierung des aktuellen Befehls, Breakpoints und Laufzeitanzeige.

2.2 Wahl der Programmiersprache

Als Programmiersprache haben wir uns für C++ entschieden. C++ erlaubt eine hardwarenahe, effiziente Umsetzung der Bit-Operationen, die für einen Mikrocontroller-Simulator allgegenwärtig sind, und unterstützt zugleich eine saubere objektorientierte Strukturierung in Klassen. Damit lassen sich die einzelnen Hardware-Bausteine des PIC, wie CPU, Speicher, Stack jeweils als eigene, klar abgegrenzte Klasse modellieren.

2.3 Wahl des GUI-Frameworks

Für die grafische Oberfläche kommt Dear ImGui zum Einsatz, ein schlankes Immediate-Mode-GUI-Framework. Die Fensterdarstellung übernimmt GLFW, das Rendering erfolgt über OpenGL. Diese Kombination wurde gewählt, weil ImGui sich besonders gut für Werkzeuge mit vielen tabellarischen Anzeigen eignet, wie etwa für die Darstellung von Registern, Speicherinhalten und Stack. Für die Quelltextanzeige mit Zeilennummern und Markern wird zusätzlich eine ImGui-basierte Texteditor-Komponente verwendet.



Architektur des Simulators

Der Simulator ist konsequent in voneinander getrennte Klassen gegliedert. Jede Klasse entspricht einem logischen Baustein des Mikrocontrollers oder einem klar umrissenen Aufgabenbereich. Dadurch bleibt der Code übersichtlich, einzelne Bausteine sind unabhängig testbar, und die Trennung zwischen Simulationslogik und Benutzeroberfläche ist sauber durchgehalten.

3.1 Schichtenarchitektur

Die Software gliedert sich in drei Schichten. Die Präsentationsschicht umfasst die Benutzeroberfläche und den Texteditor. Die Steuerungsschicht wird durch die Klasse PIC16F84 gebildet, die als Fassade die gesamte Simulation nach außen kapselt. Die Simulationsschicht enthält die eigentlichen Hardware-Bausteine: CPU, Programmspeicher, Datenspeicher und Stack. Die Benutzeroberfläche greift ausschließlich über die Fassade auf die Simulation zu und kennt die internen Bausteine nicht direkt.

Klasse	Quelldatei	Verantwortung
SimulationInterface	SimulationInterface.cpp	Fensterverwaltung, Rendering, Aufbau aller UI-Panels
Editor	ui/editor.cpp	Quelltextanzeige, Breakpoints, Steuerbuttons, Quarzfrequenz
PIC16F84	PIC16F84.cpp	Fassade: bündelt Speicher und CPU, bietet die öffentliche API
CPU	components/CPU.cpp	Befehlsdekodierung und -ausführung, Flags, Timer, Interrupts
ProgramMemory	components/ ProgramMemory.cpp	Programmspeicher, Einlesen und Parsen der LST-Datei
DataMemory	components/DataMemory.cpp	Datenspeicher (RAM/SFR) als 256-Byte-Feld
Stack	components/Stack.cpp	Acht-stufiger Hardware-Stack als Ringpuffer

Tabelle 1 – Klassen des Simulators und ihre Aufgaben.

3.2 Zusammenspiel der Komponenten

Die Fassade PIC16F84 erzeugt bei ihrer Initialisierung den Programm- und Datenspeicher und übergibt beide als Referenz an die CPU. Die CPU hält also keine eigenen Speicher, sondern arbeitet auf den gemeinsam genutzten Objekten. Der Programmspeicher wird der CPU als konstante Referenz übergeben, da die CPU während der Ausführung nur liest. Der folgende Listing 1 zeigt diesen Aufbau:

```

1  PIC16F84::PIC16F84() : programMemory(), dataMemory(),
2    cpu(programMemory, dataMemory) {
3  }
4
5  CPU::CPU(const ProgramMemory& pm, DataMemory& dm) :
6    programMemory(pm), dataMemory(dm), stack() {
7    reset();
8  }
```

Listing 1 – Ausschnitt der Konstruktoren

Ein einzelner Simulationsschritt folgt stets demselben Ablauf: Die Benutzeroberfläche ruft `step()` der Fassade auf, diese delegiert an die CPU. Die CPU holt den Befehl aus dem Programmspeicher (Fetch), dekodiert ihn, führt ihn aus und aktualisiert anschließend Timer, Watchdog und gegebenenfalls das Interruptsystem. Nach jedem Schritt liest die Oberfläche die aktuellen Register und Speicherwerte über die Fassade aus und stellt sie dar.



Programm- und Datenspeicher

4.1 Programmspeicher

Der Programmspeicher des PIC16F84 ist 1024 mal 14 Bit groß. Im Simulator wird er durch die Klasse `ProgramMemory` abgebildet. Sie verwaltet drei Felder mit jeweils 1024 Einträgen: das eigentliche Befehlsfeld, ein Belegungsfeld, das angibt, ob an einer Adresse überhaupt ein gültiger Befehl steht, und eine Zuordnung von Programmadresse zu Quelltextzeile. Letztere wird benötigt, um in der Oberfläche den gerade ausgeführten Befehl im Editor zu markieren.

Als Datentyp wurde bewusst `int` gewählt und nicht ein 14-Bit-Typ. Ein Integer wird intern ohnehin binär gespeichert, und aus dem Gemisch von Befehlscode und Argumenten lassen sich mit einfachen logischen Verknüpfungen und passenden Masken die einzelnen Bestandteile sicher heraustrennen.

4.2 Einlesen und Parsen der LST-Datei

Die Testprogramme liegen als Textdateien mit der Endung `LST` vor. Ihr Aufbau ist spaltenorientiert. Für den Simulator sind nur die ersten beiden Spalten von Bedeutung: die vierstellige hexadezimale Adresse im Programmspeicher und der vierstellige hexadezimale Befehlscode. Alle weiteren Spalten, Zeilennummer, Label oder Kommentar sind für die Ausführung nicht relevant.

Beim Einlesen wird jede Zeile geprüft: Ist sie zu kurz, um Adresse und Befehl zu enthalten, wird sie übersprungen. Andernfalls werden die beiden Teilstrings extrahiert und mit Basis 16

in Zahlen umgewandelt. Die Umwandlung ist in einen Ausnahmestapel gefasst, damit Kommentar- oder Leerzeilen, die keine gültigen Hexadezimalwerte enthalten, nicht zum Abbruch führen, sondern stillschweigend übergangen werden.

```
1  std::string address_str = line.substr(0, 4);
2  std::string command_str = line.substr(5, 4);
3
4  int address = std::stoi(address_str, nullptr, 16);
5  int command = std::stoi(command_str, nullptr, 16);
6
7  if (address >= 0 && address < 1024) {
8      memory[address] = command;
9      used[address] = true;
10     addressToLine[address] = currentLineNumber;
11 }
```

Listing 2 – Ausschnitt der Funktion zum Einlesen der LST-Datei

4.3 Datenspeicher und Bankumschaltung

Der Datenspeicher umfasst die Spezialfunktionsregister (SFR) und die allgemeinen Register (GPR). Er wird durch die Klasse `DataMemory` als Feld von 256 Byte realisiert. Lese- und Schreibzugriffe sind gegen Bereichsüberschreitungen abgesichert.

Der reale PIC ordnet seinen Datenspeicher in zwei Bänke. Die Auswahl erfolgt über das RP0-Bit im Statusregister. Die direkte Adressierung in den Befehlen ist nur 7 Bit breit, was genau dem Umfang einer Bank entspricht. Manche Register sind in beiden Bänken sichtbar. Diese sogenannte Adressspiegelung muss der Simulator korrekt abbilden, damit ein Zugriff auf das gespiegelte Register in Bank 1 dieselbe Speicherzelle trifft wie der Zugriff in Bank 0.

Hierfür dient die Methode `normalizeFileAddress()`. Sie bildet gespiegelte SFR-Adressen aus Bank 1 auf ihre Entsprechung in Bank 0 ab und faltet auch den gespiegelten GPR-Bereich zusammen. Dadurch existiert jede Speicherzelle intern nur ein einziges Mal, und es kann nicht passieren, dass über zwei verschiedene Adressen unterschiedliche Werte entstehen.

```
1  int CPU::normalizeFileAddress(int rawAddress) const {
2  int a = rawAddress & 0xFF;
3  switch (a) {           // SFR-Spiegelung Bank1 -> Bank0
4      case 0x80: return 0x00; // INDF
5      case 0x82: return 0x02; // PCL
6      case 0x83: return 0x03; // STATUS
7      case 0x84: return 0x04; // FSR
8      case 0x8A: return 0x0A; // PCLATH
9      case 0x8B: return 0x0B; // INTCON
10 }
11
12 if (a >= 0x8C && a <= 0xCF) return a - 0x80; // GPR-Spiegelung
13 return a;
14 }
```

Listing 3 – Ausschnitt der Funktion normalizeFileAddress() zur Adressnormalisierung

4.4 Zustand nach dem Reset

Beim realen Controller sind die RAM-Inhalte nach dem Einschalten undefiniert. Einige Register müssen jedoch definierte Startwerte annehmen. Der Simulator setzt sie in der Methode `reset()` der CPU. Besonders wichtig sind die TRIS-Register: Sie werden auf Eingang gestellt, damit kein I/O-Pin unbeabsichtigt als Ausgang arbeitet. Im INTCON-Register werden alle Bits gelöscht, wodurch sämtliche Interrupts zunächst gesperrt sind.

5

Der Stack

Der PIC16F84 besitzt einen acht-stufigen Stack, der ausschließlich der Speicherung von Rückkehradressen bei Unterprogrammaufrufen dient. Er kann nicht durch Befehle direkt gelesen oder beschrieben werden und wird über einen unsichtbaren Stackpointer verwaltet. Da der Stackpointer nur drei Bit umfasst, verhält sich der Stack als Ringpuffer: Nach dem achten Eintrag öffnet der Zeiger wieder auf Position null, ein neunter Aufruf überschreibt also die zuerst gespeicherte Rückkehradresse.

Im Simulator ist dieses Verhalten in der Klasse Stack umgesetzt. Beim Ablegen (Push) wird der Wert geschrieben und der Zeiger modulo acht erhöht; beim Holen (Pop) wird der Zeiger zuerst modulo acht erniedrigt und dann der Eintrag gelesen. Genau diese Reihenfolge entspricht dem realen Verhalten bei CALL beziehungsweise RETURN, RETLW und RETFIE

```
1 void Stack::push(int address) {
2     stack[stackPointer] = address;
3     stackPointer = (stackPointer + 1) % 8; // Ringpuffer
4 }
5
6 int Stack::pop() {
7     stackPointer = (stackPointer - 1 + 8) % 8; // Ringpuffer
8     return stack[stackPointer];
9 }
```

Listing 4 – Ausschnitt der Stack-Klasse mit Push- und Pop-Methoden

Die Modulo-Rechnung mit dem Summanden acht beim Pop stellt sicher, dass auch bei einem Stackpointer-Wert von null kein negativer Index entsteht. Der Stackinhalt wird in der Benut-

zeroberfläche vollständig visualisiert, sodass sich die Schachtelung von Unterprogrammen und das Ringpufferverhalten unmittelbar beobachten lassen.



Die Flags im Statusregister

Das Statusregister enthält drei Zustandsbits, die das Ergebnis einer Operation charakterisieren: das Zero-Flag (Z), das Carry-Flag (C) und das Digit-Carry-Flag (DC). Sie bilden die einzige Verbindung in die Vergangenheit, anhand derer ein Programm Entscheidungen treffen kann. Nicht jeder Befehl beeinflusst die Flags; unverändert bleiben sie, bis ein entsprechender Befehl sie anpasst.

6.1 Zero-Flag

Das Z-Flag wird gesetzt, wenn das Ergebnis einer Operation null ist, und andernfalls gelöscht. Im Simulator übernimmt das die Methode `updateZeroFlag()`, die das Ergebnis auf acht Bit maskiert und auf null prüft. Sie wird von allen Befehlen aufgerufen, die laut Befehlsübersicht das Z-Flag beeinflussen.

6.2 Carry- und Digit-Carry-Flag bei der Addition

Bei einer Addition zeigt das C-Flag den Überlauf des Acht-Bit-Zahlenbereichs an: Es wird gesetzt, sobald das Ergebnis größer als 255 ist. Das DC-Flag ist das Halbübertrags-Bit; es zeigt einen Übertrag vom unteren in das obere Nibble an, also einen Übertrag aus den Bits 0 bis 3. Der Simulator rechnet die Summe daher zunächst in voller Breite und leitet die Flags aus dieser Zwischensumme ab:

```
1  int result = oldW + literal;  
2  int result8 = result & 0xFF;  
3  wRegister = result8;  
4  
5  updateZeroFlag(result8);  
6  setStatusBit(STATUS_C, result > 0xFF);  
7  setStatusBit(STATUS_DC, ((oldW & 0x0F) + (literal & 0x0F)) > 0x0F);
```

Listing 5 – Ausschnitt Carry- und Digit-Carry-Berechnung bei der Addition

6.3 Das besondere Verhalten der Flags bei der Subtraktion

Eine Besonderheit des PIC und eine klassische Fehlerquelle ist das Verhalten von C und DC bei der Subtraktion. Der Controller führt eine Subtraktion intern als Addition des Zweierkomplements aus. Korrekterweise müsste das Carry-Bit am Ende invertiert werden; genau dieses Invertieren unterbleibt beim PIC jedoch. In der Wirkung bedeutet das: Bei einer Subtraktion wird das Carry-Flag als Borrow-Bit interpretiert.

Im Simulator ist dieses Verhalten direkt umgesetzt: Bei den Subtraktionsbefehlen wird das C-Flag über einen Größenvergleich gesetzt statt aus einem invertierten Übertrag. Für den Befehl SUBWF, der den Inhalt des W-Registers vom Fileregister abzieht, sieht das so aus:

```
1  int result = fileVal - w;  
2  int result8 = result & 0xFF;  
3  updateZeroFlag(result8);  
4  setStatusBit(STATUS_C, fileVal >= w); // Borrow-Logik  
5  setStatusBit(STATUS_DC, (fileVal & 0x0F) >= (w & 0x0F));
```

Listing 6 – Ausschnitt der Carry- und Digit-Carry-Berechnung bei der Subtraktion

Damit ergibt sich automatisch auch das vom Datenblatt beschriebene Sonderverhalten: Subtrahiert man von einer Zahl den Wert null, sind C und DC gesetzt, da der Vergleich stets erfüllt ist. Addiert man dagegen null, werden beide Flags gelöscht. Das Themenblatt weist ausdrücklich darauf hin, dass ältere Datenblätter diesen Punkt als Fehler beschrieben, neuere ihn als Borrow-Feature umdeuten.



Der Programmzähler

Der Programmzähler (PC) des PIC ist bis zu 13 Bit breit. Beim PIC16F84 mit 1024 Worten Programmspeicher werden davon zehn Bit genutzt. Nur die unteren acht Bit sind als Register PCL sichtbar; die oberen Bit lassen sich nicht direkt auslesen. Der Simulator führt den Programmzähler daher als eigenständige Variable im Hintergrund und spiegelt nach jedem Schritt nur dessen unteren Teil in das PCL-Register des Datenspeichers.

7.1 Verhalten beim Fetch-Zyklus

Nach jedem Fetch-Zyklus wird der Programmzähler um eins erhöht. Erreicht er die obere Grenze des Speichers, entsteht ein Wrap-Around: Er beginnt wieder bei null. Der Simulator bildet das ab, indem nach jeder Befehlsausführung der Programmzähler modulo 1024 gerechnet wird. Eine Berücksichtigung des PCLATH-Registers findet beim einfachen Fortschalten nicht statt.

7.2 Verhalten bei Sprungbefehlen

GOTO und CALL führen eine elfstellige Zieladresse im Befehlscode mit. Damit lassen sich Adressen innerhalb eines 2K-Blocks ansprechen. Die fehlenden oberen Bit stammen aus dem PCLATH-Register. Der Simulator setzt die Zieladresse in der Methode `buildCallGotoTarget()` zusammen: Die unteren elf Bit kommen aus dem Opcode, die Bits 3 und 4 des PCLATH liefern die oberen Bit des Programmzählers.

```
1  int CPU::buildCallGotoTarget(int instruction) const {
2      const int low11 = instruction & 0x07FF;
3      const int pclath = dataMemory.read(0x0A) & 0x1F;
4      const int high2 = (pclath & 0x18) << 8; // Bits 4:3 -> 12:11
5      return (high2 | low11) & 0x1FFF;
6  }
```

Listing 7 – Ausschnitt der Zieladressenberechnung bei CALL und GOTO

Da der Simulator einen 1K-Programmspeicher verwendet, wird die so berechnete Zieladresse anschließend modulo 1024 genommen. Bei CALL wird zuvor die Adresse des Folgebefehls auf den Stack gelegt.

7.3 Manipulation des PCL-Registers

Das PCL-Register ist Teil des adressierbaren RAM und kann mit arithmetischen oder logischen Befehlen verändert werden. Ein Schreibzugriff auf PCL bewirkt einen Sprung, denn der nächste Fetch-Zyklus holt den Befehl von der neuen Adresse. Beim Zurückschreiben wird der vollständige Programmzähler mit Hilfe von PCLATH gebildet. Der Simulator fängt jeden Schreibzugriff ab, dessen Ziel das PCL-Register ist, und ruft dann `refreshPcFromPcl()` auf:

```
1  void CPU::refreshPcFromPcl() {
2      int pcl = dataMemory.read(0x02) & 0xFF;
3      int pclath = dataMemory.read(0x0A) & 0x1F;
4      pc = ((pclath << 8) | pcl) & 0x03FF;
5  }
```

Listing 8 – Ausschnitt PCL-Manipulation

Auf diese Weise ist das in der Bewertung gesondert ausgewiesene Testprogramm zu ADDWF auf PCL mit Berücksichtigung des PCLATH korrekt abgedeckt.

8

Befehlssatz, Dekoder und Adresierung

Der PIC16F84 besitzt einen RISC-Befehlssatz. Die Befehle sind in der Regel einfach und werden schnell abgearbeitet, die meisten in vier Quarztakten, was bei einem 4-MHz-Quarz einer Befehlszeit von einer Mikrosekunde entspricht. Der Controller liest Befehlscode und zugehörige Argumente bereits im ersten Fetch-Zyklus vollständig ein.

8.1 Der Befehlsdekoder

Der Dekoder muss aus einem eingelesenen 14-Bit-Wort den Befehlstyp ermitteln. Das geschieht über Bitmasken: Der Befehl wird mit einer passenden Maske UND-verknüpft, und das Ergebnis wird mit einem bekannten Opcode-Muster verglichen. Die Opcodes und Befehlsmuster sind in der Datei `instructions.h` als Aufzählungstyp zentral hinterlegt, was den Dekoder gut lesbar hält.

Wichtig ist die Reihenfolge der Vergleiche: Befehle mit langem, spezifischem Opcode müssen vor Befehlen mit kürzerem Opcode geprüft werden, damit sie nicht versehentlich von einer allgemeineren Maske erfasst werden. Der Dekoder verwendet je nach Befehlsgruppe unterschiedlich breite Masken (etwa `0x3F00` für Literal- und Byte-Befehle, `0x3C00` für Bit-Befehle und `0x3800` für GOTO und CALL).

```
1 void CPU::decodeAndExecute(int instruction) {
2     int cycles = 1;
3     if ((instruction & 0x3F00) == Instruction::MOVLW) {
4         executeMowlw(instruction);
5         pc++;
6     }
7     else if ((instruction & 0x3800) == Instruction::GOTO) {
8         executeGoto(instruction);
9         cycles = 2;        // Sprung benötigt zwei Zyklen
10    }
11    else if ((instruction & 0x3C00) == Instruction::BSF) {
12        executeBsf(instruction);
13        pc++;
14    }
15    /* ... weitere Befehle ... */
16
17    pc %= 1024;
18    executedCycles += static_cast<uint64_t>(cycles);
19 }
```

Listing 9 – Ausschnitt des Befehlsdekoders

Jeder Befehl ist in einer eigenen execute-Methode gekapselt. Das hält den Dekoder schlank und macht die Wirkungsweise jedes einzelnen Befehls gut nachvollziehbar. Die Anzahl der verbrauchten Befehlszyklen wird mitgeführt, da Sprungbefehle und übersprungene Befehle zwei Zyklen benötigen. Dieser Wert ist die Grundlage für die Laufzeitberechnung.

8.2 Das Destination-Bit

Viele byte-orientierte Befehle besitzen ein Destination-Bit (d-Bit). Es bestimmt, wohin das Ergebnis geschrieben wird: Bei $d = 0$ landet das Ergebnis im W-Register, bei $d = 1$ im Fileregister. Der Simulator wertet das d-Bit in jeder betroffenen execute-Methode aus. Schreibt ein Befehl mit $d = 1$ in das PCL-Register, wird zusätzlich der Programmzähler neu aus PCL und PCLATH gebildet.

8.3 Unmittelbare Adressierung

Bei der unmittelbaren Adressierung folgt das Argument direkt im Befehlswort – das sind die Literal-Befehle. Da der PIC Befehlscode und Argument gemeinsam einliest, ist kein zusätzlicher Speicherzugriff nötig. Im Simulator wird die Konstante einfach mit der Maske 0x00FF aus dem Befehlswort extrahiert.

8.4 Direkte Adressierung

Bei der direkten Adressierung steht die Zieladresse als Zahlenwert im Befehl – sieben Bit breit. Zusammen mit dem RP0-Bit ergibt sich die vollständige Adresse innerhalb der gewählten Bank. Die Methode `resolveDirectAddress()` kombiniert beides und normalisiert anschließend über die bereits beschriebene Adressspiegelung.

8.5 Indirekte Adressierung

Die indirekte Adressierung nutzt das File-Select-Register (FSR) als Zeiger. Ein Zugriff auf das Register INDF an Adresse 0x00 ist kein echter Speicherzugriff, sondern wird auf diejenige Zelle umgeleitet, deren Adresse im FSR steht. Damit lassen sich zum Beispiel Summen über einen Speicherbereich elegant bilden. Der Simulator erkennt den Zugriff auf INDF und leitet ihn über den FSR-Inhalt um; ein indirekter Zugriff von INDF auf INDF liefert null bzw. bleibt wirkungslos, wie es das Datenblatt vorsieht.

```
1  int CPU::readFileRegister(int f) const {
2      int direct = resolveDirectAddress(f);
3      if (direct != 0x00) {
4          return dataMemory.read(direct);
5      }
6      // INDF: indirekter Zugriff über FSR
7      int fsr = dataMemory.read(0x04) & 0xFF;
8      int indirect = normalizeFileAddress(fsr);
9      if (indirect == 0x00) return 0; // INDF auf INDF -> 0
10
11     return dataMemory.read(indirect);
12 }
```

Listing 10 – Ausschnitt der Adressierungslogik



Die Ports und das I/O-Ausgangslatch

Der PIC16F84 besitzt zwei Ports: PORTA mit fünf nutzbaren Pins und PORTB mit acht Pins. Jeder Pin kann unabhängig als Ein- oder Ausgang betrieben werden. Dafür gibt es zu jedem Port zwei Register: das PORT-Register für die Signalpegel und das TRIS-Register für die Datenrichtung. Eine Eins im TRIS-Register macht den zugehörigen Pin zum Eingang, eine Null zum Ausgang.

9.1 Das Ausgangslatch

Eine in der Bewertung gesondert ausgewiesene Anforderung ist die korrekte Nachbildung des Ausgangslatches. Der Kern dabei: Schreibt ein Programm in das PORT-Register, obwohl die betreffenden Pins als Eingang arbeiten, geht der geschriebene Wert nicht verloren. Er wird im Daten-Latch zwischengespeichert. Wird der Pin später auf Ausgang umgeschaltet, erscheint der gespeicherte Wert sofort am Pin.

Der Simulator bildet das ab, indem er für jeden Port ein eigenes Latch-Feld führt (`portALatch`, `portBLatch`). Ein Schreibzugriff auf den Port aktualisiert immer das Latch. Welcher Wert tatsächlich am Pin sichtbar wird, entscheidet die Methode `applyPortLatchToPins()` anhand des TRIS-Registers: Für Ausgangsbits wird der Latch-Wert übernommen, für Eingangsbits bleibt der von außen anliegende Pegel erhalten.

```
1 void CPU::applyPortLatchToPins(int portAddr, int trisAddr, int latchValue)
2 {
3     const int tris = dataMemory.read(trisAddr) & 0xFF;
4     const int pins = dataMemory.read(portAddr) & 0xFF;
5     const int outputMask = (~tris) & 0xFF; // TRIS=0 => Ausgang
6     const int inputMask = tris & 0xFF; // TRIS=1 => Eingang
7     int newPins = (pins & inputMask) | ((latchValue & 0xFF) & outputMask);
8     if (portAddr == 0x05) newPins &= 0x1F; // PORTA nur 5 Bit
9     dataMemory.write(portAddr, newPins);
10 }
```

Listing 11 – Ausschnitt der Ausgangslatch-Logik

Wird das TRIS-Register geändert, ruft der Simulator dieselbe Methode erneut auf. Dadurch erscheint ein zuvor im Latch abgelegter Wert genau in dem Moment am Pin, in dem der Pin auf Ausgang umgeschaltet wird.

9.2 Simulation der Eingänge

Viele Testprogramme laufen nur dann korrekt ab, wenn an einem Eingang ein Pegelwechsel auftritt. Der Anwender kann jeden Pin per Mausklick umschalten. Damit dabei nicht versehentlich Ausgangswerte überschrieben werden, trennt die Methode `setExternalPortValue()` sauber: Von außen wirken nur Eingangsbits, Ausgangsbits bleiben von der Stimulation unberührt.

9.3 Sonderpins RA4, RB0 und RB4 bis RB7

Drei Bereiche der Ports haben Sonderfunktionen, die der Simulator berücksichtigt:

- RA4/T0CKI dient als externer Zähl Eingang für Timer 0. Der Simulator wertet hier Flanken aus, wobei die zählende Flanke über das T0SE-Bit im OPTION-Register gewählt wird.
- RB0/INT ist der externe Interrupteingang. Die wirksame Flanke ist über das INTEDG-Bit im OPTION-Register einstellbar.
- RB4 bis RB7 lösen einen Pegelwechsel-Interrupt aus. Hier ist die Flanke nicht wählbar sowohl steigende als auch fallende Flanken wirken, sofern der Pin als Eingang arbeitet.

10

Timer, Watchdog und Vorteiler

Timer, Watchdog und Vorteiler bilden eine eng verzahnte Einheit, denn der Vorteiler ist nur ein einziges Mal vorhanden und kann entweder dem Timer oder dem Watchdog zugewiesen werden. Die Zuweisung steuert das PSA-Bit im OPTION-Register.

10.1 Timer 0

Timer 0 ist ein Acht-Bit-Zählregister. Es kann entweder mit jedem ausgeführten Befehl hochzählen (interner Takt) oder externe Impulse am Pin RA4 zählen. Die Auswahl trifft das T0CS-Bit im OPTION-Register. Beim Überlauf von 0xFF auf 0x00 setzt der Timer das T0IF-Bit im INTCON-Register, das per Polling abgefragt werden oder einen Interrupt auslösen kann.

Der Simulator unterscheidet in der Methode `tickTimer0()` sauber zwischen internem und externem Takt sowie zwischen zugewiesenem und nicht zugewiesenem Vorteiler. Ein Schreibzugriff auf das TMR0-Register im selben Schritt unterdrückt das Takten, wie es das Datenblatt vorgibt.

```
1 void CPU::incrementTimer0() {
2     int tmr0 = dataMemory.read(0x01) & 0xFF;
3     int next = (tmr0 + 1) & 0xFF;
4     dataMemory.write(0x01, next);
5     if (next == 0x00) {
6         int intcon = dataMemory.read(0x0B);
7         intcon |= (1 << 2);
8         dataMemory.write(0x0B, intcon);
9     }
10 }
```

Listing 12 – Ausschnitt der Timer-Logik

10.2 Der Vorteiler

Der Vorteiler teilt die Eingangsimpulse, bevor sie das Zählregister erreichen. Die Teilerfaktoren werden über die Bits PS2 bis PS0 im OPTION-Register eingestellt. Entscheidend ist, dass die Teilerfaktoren davon abhängen, ob der Vorteiler dem Timer oder dem Watchdog zugeordnet ist.

Im Simulator liefern zwei Methoden den jeweils gültigen Teilerfaktor. Beim Timer ergibt sich der Faktor zu $2^{\text{PS}+1}$, beim Watchdog zu 2^{PS} . Der Vorteiler selbst kann weder gelesen noch direkt beschrieben werden; ein Schreibzugriff auf TMR0 oder die Befehle CLRWDT und SLEEP setzen ihn jedoch zurück.

10.3 Der Watchdog

Der Watchdog ist ein nachtriggerbares Monoflop, das den zeitlichen Ablauf des Programms überwacht. Läuft er ab, ohne durch den Befehl CLRWDT zurückgesetzt worden zu sein, erzeugt er einen Reset. Die Grundzeit beträgt 18 Millisekunden und lässt sich über den dem Watchdog zugewiesenen Vorteiler bis auf etwa 2,3 Sekunden verlängern.

Der Simulator führt den Watchdog in einem zeitbasierten Modell. In der Methode tickWdt() wird ein Mikrosekunden-Zähler entsprechend der vergangenen Befehlszyklen erhöht. Die Befehlszykluszeit hängt dabei von der eingestellten Quarzfrequenz ab. Überschreitet der Zähler die Timeout-Zeit, wird handleWdtTimeout() aufgerufen.

```
1  bool CPU::tickWdt(int cycles) {  
2      if (!wdtEnabled) return false;  
3      const int prescaler = isPrescalerAssignedToWdt() ? getWdtPrescalerDivisor() : 1;  
4      const double instructionCycleUs = 4.0 / quartzFrequencyMHz;  
5      const double timeoutUs = 18000.0 * (double)prescaler;  
6      wdtCounterUs += (double)cycles * instructionCycleUs;  
7      if (wdtCounterUs >= timeoutUs) {  
8          handleWdtTimeout();  
9          return true;  
10     }  
11     return false;  
12 }
```

Listing 13 – Ausschnitt der Watchdog-Logik



Das Interruptsystem

11.1 Auslösebedingungen

Ein Sprung zur Interrupt-Einsprungsadresse 0x0004 erfolgt nur, wenn drei Bedingungen gleichzeitig erfüllt sind: das globale GIE-Bit, das individuelle Enable-Bit der Quelle und deren Interrupt-Flag müssen gesetzt sein. Der Simulator prüft das in der Methode `shouldTriggerAnyInterrupt()` für alle Quellen.

11.2 Eintritt in die Interruptroutine

Der Sprung nach 0x0004 erfolgt in Form eines Unterprogrammaufrufs: Die Rückkehradresse wird auf den Stack gelegt. Das muss bei der Schachtelung mit normalen Unterprogrammen mitbedacht werden. Beim Eintritt löscht der Simulator das GIE-Bit und verbucht zwei zusätzliche Befehlszyklen. Das Zurücksetzen der Interrupt-Flags ist Aufgabe des Assemblerprogramms in der Interruptroutine unterbleibt es, wird die Routine immer wieder angesprungen.

```
1 void CPU::enterInterrupt() {  
2     stack.push(pc & 0x3FF); // Rückkehradresse sichern  
3     int intcon = dataMemory.read(0x0B);  
4     intcon &= ~(1 << 7); // GIE löschen  
5     dataMemory.write(0x0B, intcon);  
6     pc = 0x0004;  
7     executedCycles += 2;  
8 }
```

Listing 14 – Ausschnitt der Interrupt-Logik

11.3 Erkennung der externen Flanken

Für den externen INT-Eingang und die Pegelwechsel-Interrupts muss der Simulator Flanken an PORTB erkennen. Dazu merkt er sich den vorherigen Portzustand und vergleicht ihn vor jedem Schritt mit dem aktuellen. Für RB0 wird die wirksame Flanke über das INTEDG-Bit gewählt; für RB4 bis RB7 zählt jede Änderung eines als Eingang konfigurierten Bits. Diese Auswertung erledigt die Methode `updateExternalInterruptFlags()`.

12

EEPROM, Sleep und Laufzeit

Der PIC16F84 besitzt einen nichtflüchtigen Datenspeicher von 64 Byte. Der Zugriff erfolgt über die Register EEADR, EEDATA, EECON1 und EECON2. Ein Lesevorgang ist unmittelbar abgeschlossen. Ein Schreibvorgang dagegen benötigt eine definierte Programmierzeit von etwa einer Millisekunde und muss durch eine feste Entriegelungssequenz freigegeben werden: In EECON2 sind nacheinander die Werte 0x55 und 0xAA zu schreiben.

Der Simulator bildet beide Fälle ab. Die Entriegelungssequenz wird über eine kleine Zustandsmaschine geprüft. Ist sie korrekt, das WREN-Bit gesetzt und ein Schreibversuch über das WR-Bit angestoßen, beginnt ein zeitbasierter Schreibvorgang. Erst nach Ablauf der Programmierzeit wird das Datum tatsächlich übernommen, das WR-Bit gelöscht und das EEIF-Flag gesetzt.

12.1 Sleep-Modus

Der Befehl SLEEP versetzt den Controller in einen stromsparenden Ruhezustand. Er löscht den Watchdog-Zähler und gegebenenfalls den Vorteiler, setzt das TO-Bit und löscht das PD-Bit. Im Sleep werden keine Befehle mehr ausgeführt; die externen Interrupt-Flags und der Watchdog laufen jedoch weiter. Beendet wird der Sleep durch einen Watchdog-Timeout, einen Reset oder einen Interrupt an RB0 beziehungsweise RB4 bis RB7. Die Bits TO und PD im Statusregister geben anschließend Auskunft darüber, wie der Sleep beendet wurde.

Im Simulator behandelt die Methode `step()` den Sleep-Zustand gesondert: Statt einen Befehl zu holen, werden nur die Interrupt-Flags aktualisiert, Watchdog und EEPROM-Schreibvorgang fortgeführt und geprüft, ob ein Interrupt den Sleep beendet.

12.2 Laufzeitberechnung

Der Simulator zeigt die Zeit an, die das Programm in Echtzeit benötigen würde. Grundlage ist die Anzahl der verbrauchten Befehlszyklen, multipliziert mit der Befehlszykluszeit. Diese ergibt sich aus der Quarzfrequenz: Bei vier Quarztakten je Befehlstakt beträgt die Zykluszeit vier geteilt durch die Quarzfrequenz in Megahertz, also beispielsweise eine Mikrosekunde bei einem 4-MHz-Quarz. Die Quarzfrequenz ist in der Oberfläche frei einstellbar, was zusammen mit dem Laufzeitzähler eine in der Bewertung gesondert ausgewiesene Anforderung erfüllt.

13

Die grafische Benutzeroberfläche

Die Oberfläche ist mit Dear ImGui umgesetzt.

13.1 Aufbau der Oberfläche

Die Oberfläche gliedert sich in mehrere andockbare Panels, die jeweils einen Aspekt der Simulation darstellen oder steuerbar machen:

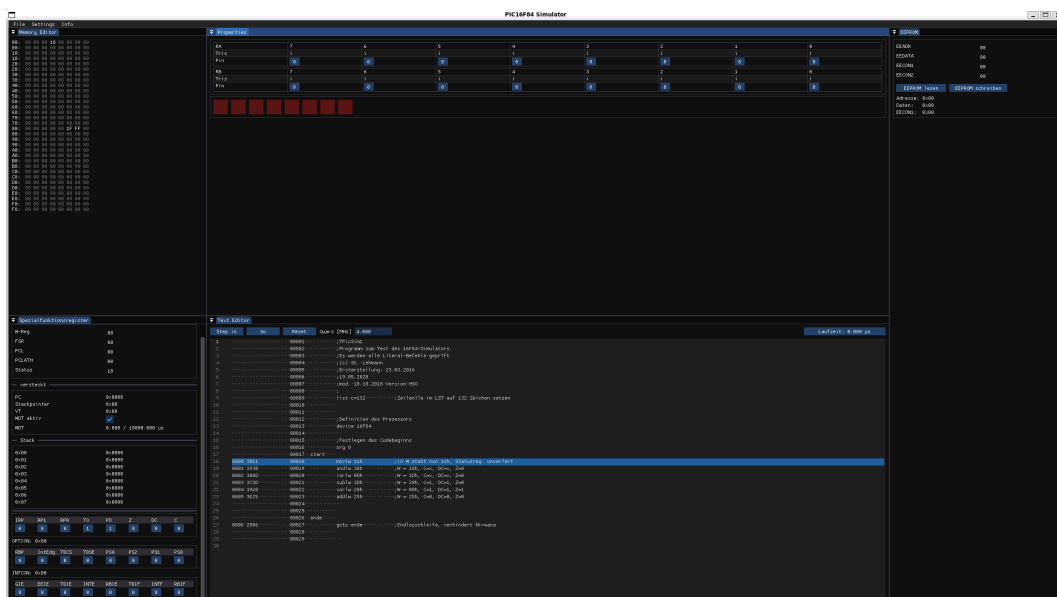


Abbildung 1 – Eine Abbildung

13.2 Steuerung der Simulation

Die Steuerung erfolgt über die Schaltflächen Step in, Go und Reset sowie über das Laden einer LST-Datei. Step in führt genau einen Befehl aus. Go startet den fortlaufenden Lauf, Reset versetzt den Controller in den Ausgangszustand. Beim fortlaufenden Lauf berechnet die Oberfläche aus der eingestellten Quarzfrequenz, wie viele Befehle je Bild auszuführen sind, sodass die Simulationsgeschwindigkeit der gewählten Frequenz folgt. Damit ein einzelnes Bild nicht blockiert, ist die Zahl der Schritte je Bild nach oben begrenzt

13.3 Breakpoints und Befehlsmarkierung

Im Texteditor kann der Anwender per Klick auf den Zeilenrand Breakpoints setzen. Realisiert ist das über einen Decorator der Editor-Komponente, der je Zeile eine unsichtbare Schaltfläche und einen roten Punkt für gesetzte Breakpoints zeichnet. Erreicht der fortlaufende Lauf eine Zeile mit Breakpoint, hält die Simulation an. Der gerade aktuelle Befehl wird zusätzlich farbig markiert; die Zuordnung von Programmadresse zu Quelltextzeile liefert dafür der Programmspeicher.

13.4 Ausgabe über das Lauflicht

Als anschauliche Ausgabe stellt das Properties-Panel den Inhalt von PORTB als Reihe von acht Leuchtelementen dar. Ein gesetztes Bit erscheint als leuchtende, ein gelöscht als dunkle Fläche. Damit lassen sich Lauflicht- und Leuchtband-Testprogramme unmittelbar beobachten.

14

Fazit

14.1 Rückblick auf das Projekt

Der PIC16F84-Simulator war für uns das bislang umfangreichste Softwareprojekt im Studium. Am Anfang stand vor allem die Frage, wie man eine so vielschichtige Aufgabe überhaupt sinnvoll zerlegt. Der Terminplan mit seinen Zwischenzielen hat uns dabei eine wertvolle Orientierung gegeben: Zuerst das Einlesen und Parsen der LST-Datei, dann der Programm- und Datenspeicher, anschließend die ersten Befehle mit einer minimalen Oberfläche und erst danach die anspruchsvollen Themen Timer, Watchdog und Interrupts. Diese schrittweise Vorgehensweise hat sich bewährt, weil wir nach jedem Abschnitt einen lauffähigen Stand hatten, an dem wir Fortschritt und Fehler unmittelbar sehen konnten.

Rückblickend war die frühe Entscheidung, jeden Hardware-Baustein als eigene Klasse zu modellieren und Simulationslogik strikt von der Benutzeroberfläche zu trennen, eine der wichtigsten. Sie hat uns später erlaubt, einzelne Bausteine unabhängig voneinander zu erweitern und zu testen, ohne den Rest des Programms zu gefährden.

14.2 Herausforderungen

Einige Teile des Projekts waren deutlich anspruchsvoller als zunächst erwartet. Besonders herausfordernd waren für uns:

- Das Sonderverhalten der Flags bei der Subtraktion. Dass der PIC das Carry- und Digit-Carry-Bit bei der Subtraktion nicht invertiert und es als Borrow-Bit interpretiert, ist eine klassische Stolperfalle. Erst durch genaues Lesen des Themenblatts und die ausdrückliche Warnung vor fehlerhaften Datenblattangaben haben wir es korrekt umgesetzt.

- Programmzähler und PCLATH. Das Zusammenspiel von PCL, PCLATH und dem unsichtbaren oberen Teil des Programmzählers, die unterschiedlich bei Fetch, bei Sprungbefehlen und bei direkter Manipulation von PCL zu durchdringen, hat einige Zeit gekostet.
- Timer, Watchdog und der eine Vorteiler. Dass es nur einen Vorteiler gibt, der je nach PSA-Bit unterschiedliche Teilerfaktoren besitzt und von verschiedenen Ereignissen zurückgesetzt wird, war logisch verzwickelt und erforderte sorgfältiges Nachdenken.
- Das Interruptsystem. Die korrekte Reihenfolge aus Flag-Erkennung, Prüfung der Enable-Bits, Sichern der Rückkehradresse und Sperren des GIE-Bits sauber in den Befehlszyklus einzufügen, war anspruchsvoll. Zumal Interrupts auch den Sleep-Modus beenden können.
- Am Ende war es auch schwer den Überblick über die vielen kleinen Details zu behalten, die das Datenblatt vorgibt. Gerade die Eigenheiten des PIC, die nicht unbedingt intuitiv sind. Dadurch ist uns erst bei der Präsentation aufgefallen das wir vergessen haben anzuzeigen, wenn ein Pin auf Input oder Output steht.

14.3 Was wir gelernt haben

Fachlich hat uns das Projekt ein sehr viel konkreteres Bild davon gegeben, was in einem Mikrocontroller tatsächlich passiert. Begriffe wie Fetch-Zyklus, Adressierungsart, Statusflag oder Bankumschaltung sind für uns von abstrakten Vorlesungsinhalten zu Dingen geworden, die wir selbst implementiert und im Detail verstanden haben. Gerade das Nachbilden der kleinen Eigenheiten des PIC, das Borrow-Verhalten, die Adressspiegelung, das Ausgangslatch hat gezeigt, wie wichtig es ist, ein Datenblatt genau und kritisch zu lesen.

Auf der methodischen Seite haben wir gelernt, ein größeres Projekt von Beginn an sauber zu strukturieren, die Arbeit aufzuteilen und uns über Schnittstellen abzustimmen. Auch der Wert von gutem Testen ist uns deutlich geworden: Die mitgelieferten Testprogramme mit ihren erwarteten Registerwerten waren ein verlässlicher Maßstab, an dem wir jeden Implementierungsschritt prüfen konnten.

14.4 Abschließende Bewertung

Es ist uns gelungen, einen vollständig funktionsfähigen Simulator zu entwickeln, der den gesamten Befehlssatz, das Speicher und Registermodell, Timer, Watchdog und Vorteiler, das Interruptsystem, das EEPROM mit Programmierzeit sowie das I/O-Ausgangslatch korrekt nachbildet und über eine übersichtliche, frei konfigurierbare Oberfläche bedienbar macht. Alle Testprogramme werden mit den erwarteten Ergebnissen durchlaufen.